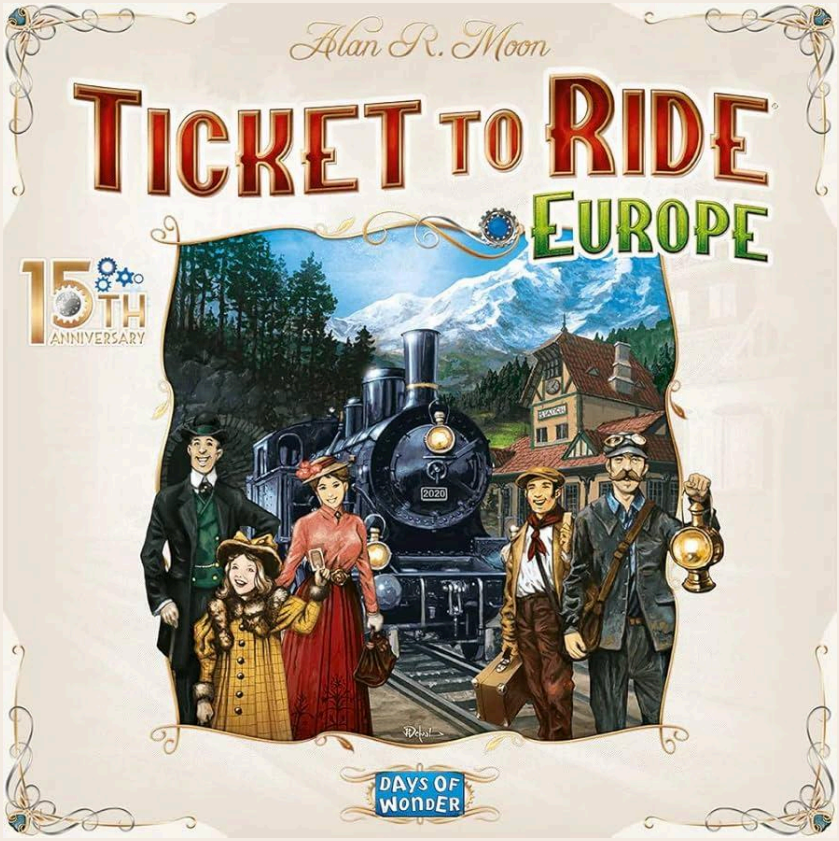


Ticket To Ride Europe Prospectus

3/21/25

Presented by:

Justin Wei

Dion Hadri

Nimay Harish

Swarnim Ghimire

Kanishk Baviskar

Table Of Contents

0 Summary **3**
1 Time Management and Team Communication **3**
1.1 Due Dates 3
1.2 Milestones 4
1.3 Time Tables 6
1.4 Team Communication and Management 19
1.5 Code Management 20
2 UML Diagrams and Discussion **21**
2.1 Frontend UML 21
2.2 Backend UML 26
3 Game Algorithm **31**
3.1 Open the game 31
3.2 Set up game 31
3.3 Game loop (repeats until gameOver is true) 33
3.4 Game End 38
4 Cards, Tickets and Map Creation Algorithm **38**
5 Scoring Algorithm **39**
5.1 During the game 39
5.2 At the end of the game 39
5.3 If there is a tie 40
6 GUI Discussion **40**
6.1 Start the game 41
6.2 Play the game 43
6.3 End of the game 52
7 Program Testing Methodology and Test Data **53**
7.1 Early stages/Game setup: 53
7.2 Start of Gameplay Setup 54
7.3 Gameplay Testing 54
7.4 End of Game 55
7.5 Test Data 55
8 Foreseeable Problems and Issues **58**
8.1 Game Map Graphics 59
8.2 Monitor Player’s Mouse Click 59
8.3 Locomotive Incorporation 59
8.4 Check Ticket is Completed or Not 60
8.4 Handling Multiple Players 60
8.5 Game End Conditions 60

8.6 Debugging Challenges 61

0 Summary

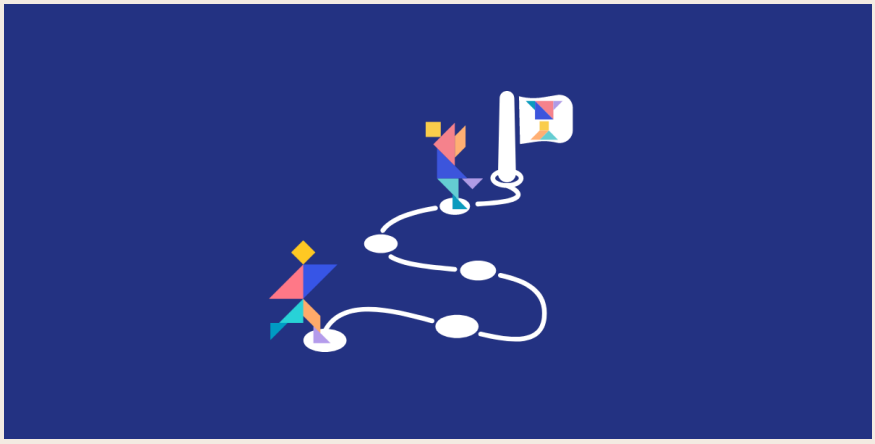
This is the prospectus for our "Ticket to Ride Europe" game in Java, developed by our team including Justin Wei, Dion Hadri, Nimay Harish, Swarnim Ghimire and Kanishk Baviskar. This prospectus is a brief document that outlines the purpose, scope, and structure of our Ticket to Ride Europe Java program before we write the actual code in the following six weeks. The prospectus serves as our planning document that helps us organize our thoughts, define key components, lay out the program frame and ensure the code meets the requirements. Besides the technical parts, the prospectus also outlines our time management, weekly activities, team communications to help us work as a team towards the final product.

1 Time Management and Team Communication

1.1 Due Dates

- **2/28-** Finish rough outline of the project
- **3/6-** Begin framing of front end and back end UMLs
- **3/21-** Hard due date for prospectus
- **3/25** - presentation of the prospectus
- **4/10-** Soft due date: finish the functional part of the program
- **4/21-** Soft due date: finish the aesthetics of graphic user interface
- **5/2-** Hard due date for the entire project

1.2 Milestones



1. Ticket To Ride Europe Project Proposal

    - Create and complete a plan of our approach and design for the game

2. Create skeleton for essential Java classes

    - Structure the basics of our classes and set an outline for us to fill out later

3. Create all the data and images for the Project

    - Design the all the cards, card backs, tickets, as well as the front cover and background, train stations, and the game board

4. Create Panels and Graphics

    - Display everything through game panel and design graphics

5. Implement Gameplay Mechanics

    - Allow the player to execute actions such as drawing cards, placing tracks, or train stations, etc.

6. Final GUI Connections

- Connect GUI to values stored by players (score, tickets, cards, trains, longest route, train stations, etc.)

- Implement buttons and score displays (Display active longest train, number of train stations, number of trains, tickets, and active deck of cards)

7. End Game Conditions

- Determines when the game will end (two or less trains left)

8. Final Score Calculation

- Calculate scores and leaderboard (ticket completion + game points + longest train)

9. Finish Game Start and End Panels

- Start panel with options to review game rules or to start play the game. End panel to show players scores and winning player, and allow them to play the game again.

10. Test Program and Fix Bugs

- Test the game and fix any bugs that are uncovered

11. Final Game Submission

- Submit our game

1.3 Time Tables

The following time table lists suggested dates of completions of milestones/goals.

Week	Goals
Week 1: 2/17-2/21	☐ Learn how to play Ticket to Ride Europe ☐ Establish a form of communication ☐ Start drafting prospectus
Week 2: 2/24-2/28	☐ Assign and document game development tasks ☐ Create time tables ☐ Brainstorm potential issues and fixes
Week 3: 3/3 - 3/7	☐ Begin frontend and back-end UMLs' ☐ Begin writing game logic and overall structure ☐ Begin designing a model GUI (start screen and end screen)
Week 4: 3/17 - 3/21	☐ Collect graphics (ex. trains, cards, train stations, and ticket pictures) ☐ Write descriptions for UI layout ☐ Finalizing prospectus
Week 5: 3/24 - 3/28	☐ Finish polishing prospectus ☐ Begin and finish the slideshow needed for the presentation ☐ Prepare presentation topics
Week 6: 3/31 - 4/4	☐ Begin coding core classes

	☐ Start on all three game screens ☐ Begin front end graphics work
Week 7: 4/7 - 4/11	☐ Continue working on the front end graphics and screens ☐ Implement all of the core classes correctly ☐ Finalize game screen
Week 8: 4/14 - 4/17	☐ Finish up screens ☐ Continue work on basic core classes necessary for the game ☐ Finalize panels
Week 9: 4/22 - 4/25	☐ Add features to game if core classes are complete ☐ Finish Core classes ☐ Finish all the panels
Week 10: 4/28 - 5/2	☐ Play test our game and make sure everything works correctly ☐ Debug any mistakes ☐ Fix all the small errors
Week 11: 5/5 - 5/9	☐ Add additional features to UI if time permits ☐ Add more screens if time permits

Week 1	Monday 2/17	Tuesday 2/18	Wednesday 2/19	Thursday 2/20	Friday 2/21
Nimay	-Learn the game rules of Ticket to Ride Europe	-Find or create a good title picture for the prospectus	-Tweak the time tables	-Finish time table	-Work on peer review -Add to the prospectus
Dion	-Learn the game rules of Ticket to Ride Europe	-Start brainstorming different layouts for the prospectus	-Tweak the time tables	-Finish time table	-Add to the peer review -Figure out format of the prospectus
Justin	-Learn the game rules of Ticket to Ride Europe	-Work on the contents and the table of contents of the prospectus	-Tweak the time tables	-Finish time table	-Contribute opinions to peer review -Finish the table of contents
Kanishk	-Learn the game rules of Ticket to Ride Europe	-Critique the small details and edit grammar/layout mistakes	-Work on the time table	-Finish time table	-Discuss the peer review -Add in all the tables and timeframes
Swarnim	-Learn the game rules of Ticket to Ride Europe	-Start brainstorming for the UML	-Tweak the time tables	-Finish time table	-Contribute opinions to peer review -Finish the table of contents

Week 2	Monday 2/24	Tuesday 2/25	Wednesday 2/26	Thursday 2/27	Friday 2/28
Nimay	-Edit the bullet points and plan	-Polish the prospectus	- Graph Test	-Start programming the project	-Contribute to peer reviews
Dion	-Edit the bullet points and plan	-Polish the prospectus	- Graph Test	-Start programming the project	-Add to the peer review
Justin	-Start planning the code and classes	-Polish the prospectus	- Graph Test	-Start programming the project	-Work on peer review
Kanishk	-Brainstorm graphics and classes/code	-Polish the prospectus	- Graph Test	-Start programming the project	-Discuss peer review
Swarnim	-Plan the code and classes necessary for the game	-Polish the prospectus	- Graph Test	-Start programming the project	-Ponder about peer review

Week 3	Monday 3/3	Tuesday 3/4	Wednesday 3/5	Thursday 3/6	Friday 3/7
Nimay	-begin thinking about front and back end uml	-work on creating cards for Ticket to Ride Europe	-SAT	-work on creating cards for Ticket to Ride Europe	-Contribute to peer reviews
Dion	-look through the prospectus for any small issues that may have been missed	-work on creating cards for Ticket to Ride Europe	-SAT	-work on creating cards for Ticket to Ride Europe	-Add to the peer review
Justin	-brainstorm the front and back end uml for the game	-work on creating cards for Ticket to Ride Europe	-SAT	-work on creating cards for Ticket to Ride Europe	-Work on peer review
Kanishk	-begin final touch ups for the prospectus	-work on creating cards for Ticket to Ride Europe	-SAT	-work on creating cards for Ticket to Ride Europe	-Discuss peer review
Swarnim	-brainstorm the front and back end uml for the game	-work on the front end	-SAT	-continue work on front end	-Discuss peer review

Week 4	Monday 3/17	Tuesday 3/18	Wednesday 3/19	Thursday 3/20	Friday 3/21
Nimay	Finish up on potential bugs and their respective solutions	Look through for any potential grammar mistakes in the wording	Polish the prospectus and edit any small errors	Polish the prospectus and edit any small errors	Polish the prospectus and edit any small errors
Dion	Finish up on potential bugs and their respective solutions	Look through for any potential grammar mistakes in the wording	Polish the prospectus and edit any small errors	Polish the prospectus and edit any small errors	Polish the prospectus and edit any small errors
Justin	Add to the prospectus and finish the UML diagrams, front and back end, and also user interface.	Polish the last bits of the prospectus and revise any small errors that are found in the prospectus	Polish the last bits of the prospectus and revise any small errors that are found in the prospectus	Polish the last bits of the prospectus and revise any small errors that are found in the prospectus	Polish the last bits of the prospectus and revise any small errors that are found in the prospectus
Kanishk	Finish up on potential bugs and their respective solutions	Look through for any potential grammar mistakes in the wording	Polish the prospectus and edit any small errors	Polish the prospectus and edit any small errors	Polish the prospectus and edit any small errors
Swarnim	Back end UML	Work on the UML diagrams	Work on the UML diagrams	Work on the UML diagrams	Polish the prospectus

Week 5	Monday 3/24	Tuesday 3/25	Wednesday 3/26	Thursday 3/27	Friday 3/28
Nimay	Polish the prospectus	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation
Dion	Polish the prospectus	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation
Justin	Polish the prospectus	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation
Kanishk	Polish the prospectus	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation
Swarnim	Polish the prospectus	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation	-present /listen to presentation

Week 6	Monday 3/31	Tuesday 4/1	Wednesday 4/2	Thursday 4/3	Friday 4/4
Nimay	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code
Dion	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code
Justin	Begin the graphics work on the front end. Start writing the panel classes	Begin the graphics work on the front end. Start writing the panel classes	Begin the graphics work on the front end. Start writing the panel classes	Begin the graphics work on the front end. Start writing the panel classes	Begin the graphics work on the front end. Start writing the panel classes
Kanishk	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code	Start the implementation of basic backend classes in our code
Swarnim	Begin the graphics work on the front end. Start writing the panel classes	Begin the graphics work on the front end. Start writing the panel classes	Begin the graphics work on the front end. Start writing the panel classes	Begin the graphics work on the front end. Start writing the panel classes	Begin the graphics work on the front end. Start writing the panel classes

Week 7	Monday 4/7	Tuesday 4/8	Wednesday 4/9	Thursday 4/10	Friday 4/11
Nimay	Begin writing some of the more challenging portions of our backend game	Begin writing some of the more challenging portions of our backend game	Begin writing some of the more challenging portions of our backend game	Begin writing some of the more challenging portions of our backend game	Begin writing some of the more challenging portions of our backend game
Dion	Help out with the game panels and screens	Help out with the game panels and screens	Help out with the game panels and screens	Help out with the game panels and screens	Help out with the game panels and screens
Justin	Write core classes for our backend uml that require a heavy workload	Write core classes for our backend uml that require a heavy workload	Write core classes for our backend uml that require a heavy workload	Write core classes for our backend uml that require a heavy workload	Write core classes for our backend uml that require a heavy workload
Kanishk	Begin writing some of the more challenging portions of our backend game	Begin writing some of the more challenging portions of our backend game	Begin writing some of the more challenging portions of our backend game	Begin writing some of the more challenging portions of our backend game	Begin writing some of the more challenging portions of our backend game
Swarnim	Help out with the game panels and screens	Help out with the game panels and screens	Help out with the game panels and screens	Help out with the game panels and screens	Help out with the game panels and screens

Week 8	Monday 4/14	Tuesday 4/15	Wednesday 4/16	Thursday 4/17	Friday 4/18
Nimay	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game
Dion	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game
Justin	Finish up the challenging and intense classes for our back end code and start to wrap up front end panel as well	Finish up the challenging and intense classes for our back end code and start to wrap up front end panel as well	Finish up the challenging and intense classes for our back end code and start to wrap up front end panel as well	Finish up the challenging and intense classes for our back end code and start to wrap up front end panel as well	Finish up the challenging and intense classes for our back end code and start to wrap up front end panel as well
Kanishk	Work on finalizing the panels and screens for our UI and game	Work on finalizing the panels and screens for our UI and game	Work on finalizing the panels and screens for our UI and game	Work on finalizing the panels and screens for our UI and game	Work on finalizing the panels and screens for our UI and game
Swarnim	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game	Keep working on the basic core classes necessary for our game

Week 9	Monday 4/21	Tuesday 4/22	Wednesday 4/23	Thursday 4/24	Friday 4/25
Nimay	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding
Dion	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding
Justin	Implement all of the necessary portions of logic and classes for our code to work	Finish up the back end classes	Continue work on the front end uml and graphics panel	Continue work on the front end uml and graphics panel	Continue work on the front end uml and graphics panel
Kanishk	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding	Work on the backend core classes and finish up all of the simple coding
Swarnim	Work on the graphics and user interface portion of our code	Work on the graphics and user interface portion of our code	Work on the graphics and user interface portion of our code	Work on the graphics and user interface portion of our code	Work on the graphics and user interface portion of our code

Week 10	Monday 4/28	Tuesday 4/29	Wednesday 4/30	Thursday 5/1	Friday 5/2
Nimay	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code
Dion	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code
Justin	Finish the core classes and game panels	Help debug anything wrong with the code	Look for errors in the creation and writing of our code	Start looking for errors and debugging the code	Start looking for errors and debugging the code
Kanishk	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code
Swarnim	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code	Start looking for errors and debugging the code

Week 11	Monday 5/5	Tuesday 5/6	Wednesday 5/7	Thursday 5/8	Friday 5/9
Nimay	Add any finishing touches	Add any finishing touches	Add any finishing touches	Add any finishing touches	Add any finishing touches
Dion	Add any finishing touches	Add any finishing touches	Add any finishing touches	Add any finishing touches	Add any finishing touches
Justin	Add small user features if time permits	Add small user features if time permits	Add small user features if time permits	Add small user features if time permits	Add small user features if time permits
Kanishk	Add any finishing touches	Add any finishing touches	Add any finishing touches	Add any finishing touches	Add any finishing touches
Swarnim	Add any finishing touches	Add any finishing touches	Add any finishing touches	Add any finishing touches	Add any finishing touches

1.4 Team Communication and Management

We'll have an iMessage group chat to communicate with each other regarding project coordination and inquiries. We will have a short group call every Friday to recap newly finished tasks, coordinate efforts for the following week accordingly, give each member a final peer review score, and discuss any ongoing or foreseeable issues.



Each member will have a weekly list of tasks to complete each week. There will be a log for each member showing what they completed over the course of the week. Any undone tasks will either be completed over the weekend by the assigned member or completed by another member. Depending on the situation, a member may receive a weekly peer review score deduction. An example of a person not receiving a deduction would be if they were sick.



1.5 Code Management

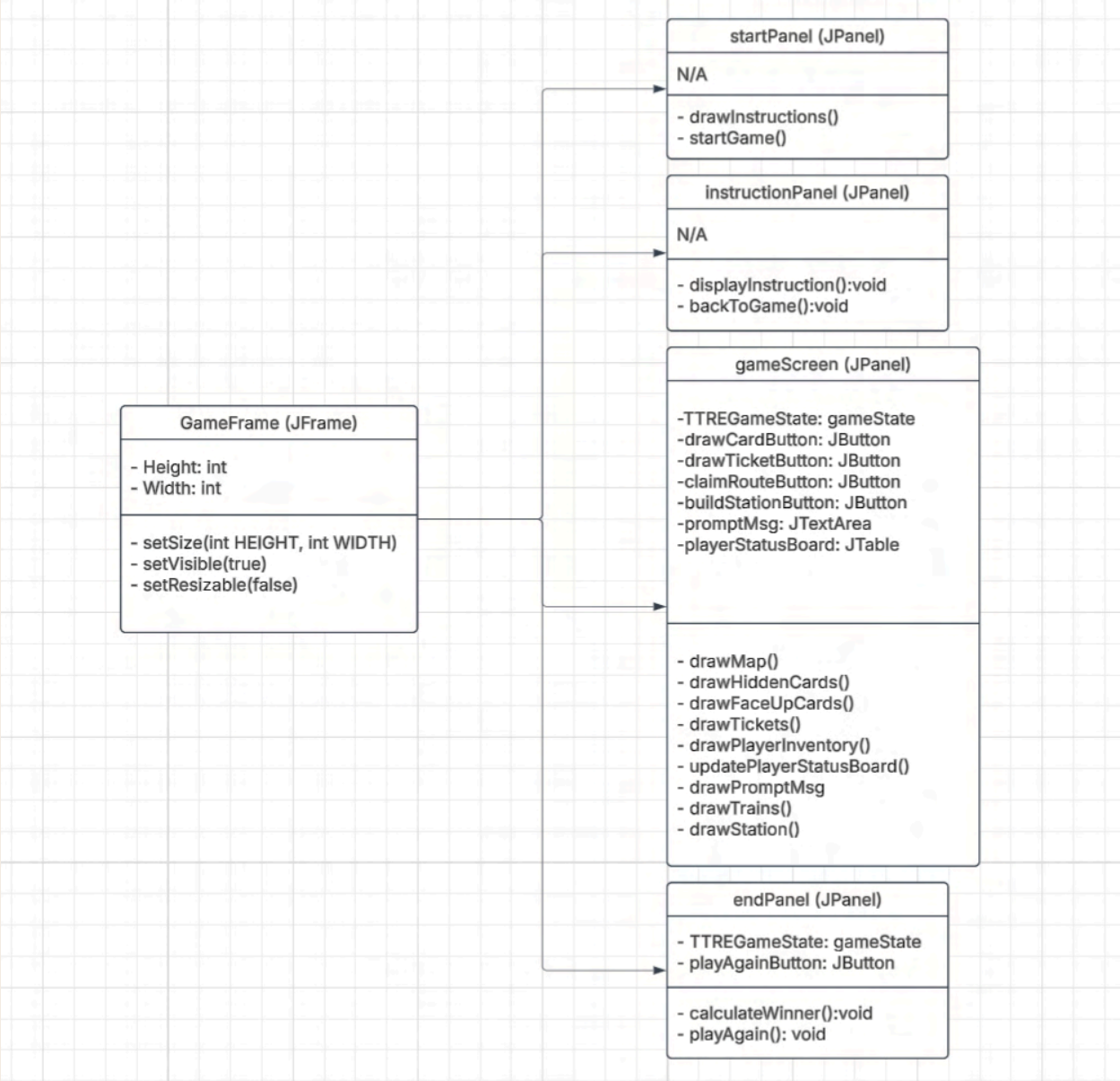
Our team will use GitHub to share code with other group members. We will create a repository and each member will have their own branch. Each member can push code into the main branch after another member checks the code for errors.



2 UML Diagrams and Discussion

We have a frontend UML and a backend UML. The frontend UML manages the graphic user interface, while the backend UML maintains the game logic.

2.1 Frontend UML



We decided to keep our frontend simple by having one major frame (gameFrame) that displays the whole user interface. Under gameFrame, there are four other panels: startPanel, endPanel, instructionPanel, and gameScreen.

We will use KeyboardListener and MouseListener in Java to handle user input events, as part of the graphic user interface.

- 4 JButtons to show the 4 player's move options (draw card, draw ticket, claim route, build station) so the player can click the button to select his/her move.
- Cities: we initially thought about using a JButton for each city so the program can register which city the player clicks. However, this would introduce many buttons as there are 46 cities in the game. Using an individual JButton for each city would become cumbersome. Instead, we will associate each city with a (x, y) Point using Java Point class. The (x, y) coordinates describe each city's graphical location on the map. Then we will use MouseListener to monitor the mouse click. If a mouse click is within a certain distance of the city's (x, y) location, it means the player clicked/selected a city.
- House cards, house tickets: we will use MouseListener to track mouse clicks. Similar as for cities, we will use the mouse click location to decide what the player clicks and what game logic to follow.

startPanel

This panel is the first panel for the user. It allows the user to select between "Start Game" and "Game Rules". The "Start Game" button, once clicked, will take the user to the main gameScreen to start the

game. The "Game Rules", once clicked, will displace the game instructions.

- drawInstructions: paints the image of the instructions on the screen
- startGame: start playing the game

gameScreen

This is where the main game user interface is. This panel will be the main connection to the back-end logic as it draws and updates what to be displayed on the game screen, including gamp map, player cards, house cards, tickets, and player status board. It also displays prompt messages to give players hints on what to do next, i.e, to click on train cards, or click on cities to claim routes.

gameScreen will have the following main properties to collect user inputs on the screen and then pass that information to the back-end logic in gameState class.

- TTREGameState: gameState (this is the connection with the back-end logic)
- drawCardButton, drawTicketButton, claimRouteButton, buildStationButton: JButton
- promptMsg: JTextArea, this is to display the prompt message box which shows who the current player is and gives the player tips on what to do next. Also, it shows what cities he/she has selected, and if route claiming or station building is successful or not.
- playerStatusBoard: JTable, this is to display all of the players' scores, colors, and longest routes during each step of the game.

The following methods are included:

- drawCards, drawTickets: draw the house deck train car cards and house destination tickets. The house deck train car cards will have 5 face up and the rest hidden.
- drawMap: this is to draw all the cities and the routes in between them. Once a player claims a route, the claimed route will be labelled with the player's name to make the route ownership visible on the gameboard.
- drawPlayerInventory: this is to draw the cards, tickets, stations inventory of the current player, and also to display his/her current score.
- updatePlayerStatusBoard: this is to show the real time score and longest continuous route for each player.
- drawPromptMsg: this is to display messages to the current player during his/her turn. The intention is to guide the player on the game user interface. It displays what he/she needs to do, for example, clicking a city, or clicking house cards to draw cards, or clicking house tickets to draw tickets. It also displays interactive information. For example, if a player clicks on city "Paris" when he/she is trying to claim a route, the prompt message would be "You have selected Paris as part of the claimed route". This tells the player that his/her click on the city "Paris" is taken in by the game.
- drawTrains: this is to draw a player's colored trains when he/she claims a route.
- drawStation: this is to draw a player's station at a city when he/she builds a train station there.

endPanel

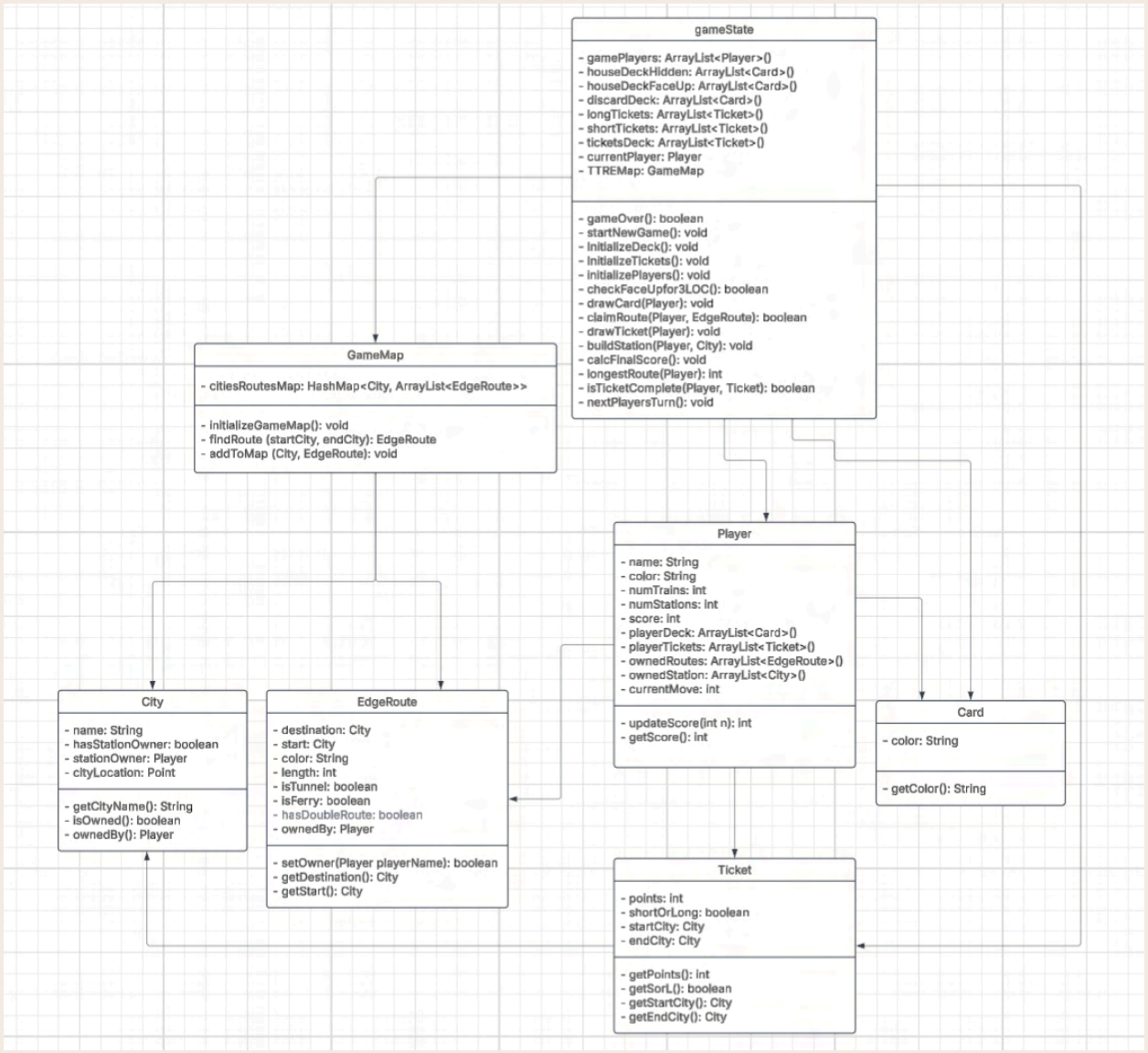
This panel displays each player's final scores and the winner of the game.

- calculateWinner: calls calcFinalScore from the gameScreen and displays the scoring table on the screen with the winner displayed at the top.
- playAgain: to play the game again.

instructionPanel

This panel displays the game instructions in case the player wants to refresh the game rules.

2.2 Backend UML



For the backend logic of the game, we decided to keep it simple using 7 classes representing the 7 main components of the game: gameState, gameMap, player, card, city, edgeRoute, and ticket.

GameState

This is the main brain and functionality of the game. It has the following properties: gamePlayers, gameState, houseDeckHidden, houseDeckFaceUp, discardDeck, longTickets, shortTickets, ticketsDeck, currentPlayer and gameMap.

It has the following methods:

- gameOver: check if any player has <= 2 trains remaining after each play step and returns true if so.
- intializeDeck: adds all of the proper train cards into the deck
- intializeTickets: adds all the destination tickets into a deck
- intializePlayers: gives players their correct attributes (ex. cards, trains, stations, tickets)
- checkFaceUpFor3LOC: checks the face up cards for 3 locomotive cards.
- claimRoute(Player, EdgeRoute): this is the method to call when the player wants to claim a route at his/her current turn. The method checks if the player has the correct number of cards and the correct colors. If the player can claim the route, the method updates the edgeRoute's in the gameMap to reflect the player's ownership of the route. If the player doesn't have the right amount of train cards or colors, the method will return false.
- drawCard(Player): this is the method for the current player to draw 1 or 2 cards from the house deck.
- drawTicket(Player): this is the method for the current player to draw destination tickets from the tickets deck.

- buildStation(Player): this is the method for the current player to build a train station at his/her chosen city.
- calcFinalScore(): this is the method when the game is over, to calculate the final scores for each player.
- longestRoute(): this is the method to update the longest route number for each player at every turn. So the information is shown at the game leaderboard. This method searches the gameMap for routes owned by each player and returns the length of his/her current longest continuous route.
- isTicketComplete(Player, Ticket): this method checks if the Player has completed the Ticket. It starts from the start city and searches through the routes towards the end city, checking if the player owns all the routes or has a train station if he/she doesn't own the route. Returns true if the ticket is completed by the player.

GameMap

This class is where the game board map including cities and their routes is stored. The gameMap is implemented using HashMap. In the HashMap, each city is the node. All the adjacent cities of that city are stored in a list of EdgeRoutes.

- initializeGameMap: initialize the starting map at the very beginning of the game, with all cities and routes information
- findRoute: finds the route(s) between the start and end city
- addToMap: adds a city and a route to the map

Player

This class represents each player in the game and their respective "possessions" such as stations, trains, tickets, routes claimed and latest score.

- updateScore: adds to the score of the player after each turn
- getScore: returns the score of the player's points

Card

This class represents a train card in the game and their respective colors. Each card's properties will be assigned during game initialization.

- getColor: gets the String color of the desired card

Ticket

This class represents a ticket in the game and their number of points. Similar to the Card class, each ticket's attributes, points, short or long, start city and end city, will be assigned during game initialization.

- getPoints: gets the int number of points on the destination ticket
- getSorL: gets the boolean value if true then it is a short route and if false it is a long route
- getStartCity: returns the start city of the destination ticket
- getEndCity: returns the end city of the destination ticket

City

This class represents each city in the game and if the city has a station owner and if so, owned by which player. The city also has a (x, y) coordinates which are used for graphic purposes. The following are the methods:

- getCityName(): returns the name of the city
- isOwned(): returns true if the city already has a station owner
- ownedBy(): returns the player who owns the station

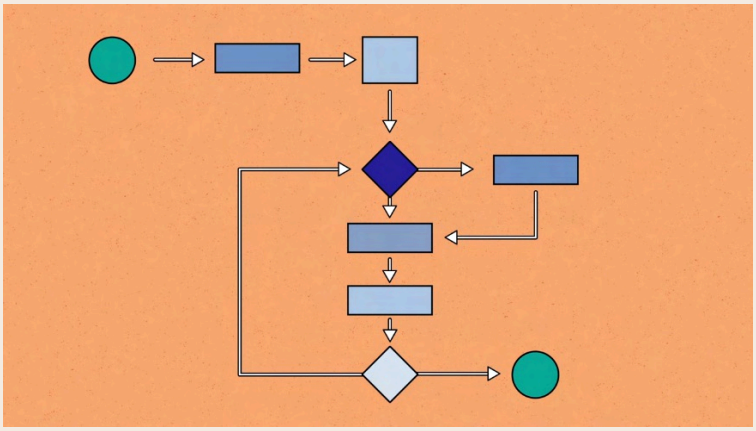
EdgeRoute

This class represents each edge route in the map between two cities. It has the following properties: destination city, start city, color of the route, length of the train cars, if it is a tunnel or ferry, the player who owns the route if any.

- setOwner(Player): sets the player to the route that they placed
- getDestination: returns the end city of the edge route
- getStart: returns the starting city of the edge route

3 Game Algorithm

The following is the entire algorithm for the game. The game's program will follow these basic steps.



3.1 Open the game

a. Create and instantiate the start screen and display the start screen (startPanel) which includes game instructions.

3.2 Set up game

a. Create and initialize 158 train cards and their properties (colors). Locomotives are treated as a special color "LOC". Each color is given a 3 letter name, "RED", "GRE", "BLU", "BLA", "ORA", "PUR", "WHI", "YEL".
b. Create and initialize 46 Destination Tickets and their properties: short(regular) route or long route, starting city, ending city and the associating number of points.
c. Create the map. This is the gameMap, the main map which holds the information about the cities and routes. The gameMap will be initialized before starting the game. The route information is stored in "EdgeRoute".

d. Create and initiate 4 players and their attributes. Either assign each player a name or let players input their game names. Assign or pick player colors. Each player is given 45 train cars (numTrains = 45), 3 same color train stations (numStations = 3). At the start, each player's score is set to 0. Each player's ownedRoutes is set to blank as the player has not claimed any route yet. The player's currentMove is set to 0. currentMove could be 1/2/3/4 depending on which option the player chooses at his/her turn.
e. Shuffle the train cards and deal 4 cards to each player to start the game. Add the 4 cards to the player's playerDeck arraylist. Remove those 4 cards from houseDeckHidden.
f. Turn the top 5 train car cards of the remaining deck face up (show them on the game screen). The 5 face up cards are stored in houseDeckFaceUp arraylist. The rest of the train car cards remain in the houseDeckHidden arraylist and the cardback image is displayed to show the hidden deck. Then call the CheckFaceUpfor3LOC() method to check whether or not there are 3 locomotives in the 5 face up cards. If the method returns true, discard all 5 face up cards to the discardDeck arraylist and replace them with 5 new cards from the houseDeckHidden arraylist.
g. Shuffle the 6 long route destination ticket cards and deal 1 ticket to each player. The 1 ticket will be added to playerTickets arraylist. It will also be removed from the house's longTickets arraylist.
h. Shuffle the 40 short(regular) route destination ticket cards and deal 3 tickets to each player. The 3 tickets will be added to

playerTickets arraylist. It will also be removed from the house's shortTickets arraylist.

i. Then the player will have the opportunity to choose which two destination tickets they want to keep. A game screen will be displayed allowing the player to select which two they keep. Then the two tickets that they chose will remain in their playerTickets arraylist and the other two destination tickets will be returned to the shortTickets arraylist or longTickets arraylist depending on what type of destination ticket it is.

3.3 Game loop (repeats until gameOver is true)

A player has 4 options during his/her turn: draw train car cards, draw destination tickets, claim a route, or build a train station. 4 buttons will be displayed on the screen so that the player can select which option he/she wants to go with. Each of the option is described in more details as follows:

I. Player clicks "draw train car cards" button:

- Set player's currentMove as 1. Display "draw train car cards" as player's move so it's clear.
- The player draws a train car card by clicking on the card. The player will draw 2 train cards from the houseDeck (either face up or hidden pile), or just 1 card if he/she picks a face up locomotive card (color = "LOC"). The 1 or 2 cards drawn are removed from the houseDeck and added to the player's playerDeck.
- If the player draws 1 card from houseDeckFaceUp, move 1 card from houseDeckHidden to houseDeckFaceUp. I.e., the houseDeckFaceUp should remain at 5 cards at all times.

- Call CheckFaceUpfor3LOC() method: this method checks if the 5 face up cards have 3 Locomotives. If so, the 5 face up cards will be dismissed (i.e., added to discardDeck arraylist) and new 5 cards will be drawn as the houseDeckFaceUp.
- Whenever the draw pile (houseDeckHidden) is empty, the discardDeck should be shuffled into a new draw pile, i.e., it becomes the new houseDeckHidden. Reset the discardDeck to be empty.
- Rare case: if there is no card left to be drawn from the deck, the player will not be allowed to take the "draw train car card" option, and will have to follow any of the other 3 options.
- No change to player's score.
- Refresh gameScreen so it shows the updated player status and deck cards.
- Move to the next player's turn.

II. Player clicks "claim a route" button:

- Set player's currentMove as 2. Display "claim a route" as the player's move so it's clear.
- The player needs to select/click the same color train cards from his/her cards, the ones that he/she wants to use to claim the route. The selected cards would be pulled up from the player's deck. "LOC" locomotives can be treated as any color for route claiming purposes. If the player selects/clicks different color train cards, display a red warning message on the gameScreen "Please select the same color train cars". The player can deselect the card by

clicking it again, i.e., put the selected card back to his/her deck.

- The player then needs to select the route he/she wants to claim, by clicking on the map the starting city and ending city along the route.
- The program then calls findRoute (startCity, endCity) method. The method searches through gameMap hashMap to find the routes between the starting city and ending city.
- If the route has the correct color and length, and is not claimed (i.e., ownedBy is blank), the player can claim the route. The program calls for claimRoute method which does the following:
    - Watch for the ferry and tunnel special cases. Ferry route: make sure the player has the correct number of "LOC" locomotive cards required to claim the ferry route. Tunnel route: after making sure the player has the correct cards to claim the tunnel route, the next step is to flip 3 cards from the houseDeckHidden arraylist of cards. Then check for any matching cards between the color that the player played and the 3 cards drawn from the houseDeckHidden arraylist. The player will need to add additional cards of the same color based on however many cards from the deck match the color of card they played. If they do not have the necessary amount of cards, nothing will happen on their turn and they will effectively be skipped.

    - Set EdgeRoute as ownedBy this player. Add the claimed route to the player's ownedRoutes list. Display the cars along the route with the player's name on them on gameScreen. So the claimed route is visible to all players in the game.
    - Player's number of trains to be reduced by the route length. Player's score to be updated.
    - Display "Congratulations Player X, Route Claimed, xx points added!" in the gameScreen.
- If the route color or length does not match with what the player has, or the player already owns 1 of the 2 routes between the cities, the player cannot claim the route. See game rule "double routes": if there are double (parallel) routes between the cities and one route has already been claimed by the same player, he/she cannot claim the 2nd route of the double route.
    - Display "The route cannot be claimed. Please reselect train cards" in the gameScreen. Go back to the beginning of this "claim a route" step. The player can restart and select another set of train cards to claim a route.
    - Optional: allow the player the option to jump out of "claim a route" if he/she wants to draw cards instead, i.e., go back to the beginning of "draw train car card" option.

III. Player clicks "draw destination ticket" button

- Set player's currentMove as 3. Display "draw destination ticket" as player's move so it's clear.
- Player draws 3 new cards from ticketsDeck. If the ticketsDeck has less than 3 cards, the player only draws whatever cards the ticketsDeck has.
- Players decide to keep 1, 2, or all 3 of the ticket cards. For the tickets he/she chooses to keep, add them to the player's playerTicket arraylist. For the tickets he/she chooses not to keep, add those to the bottom of the ticketsDeck.

IV. Player clicks "build train station" button

- Set player's currentMove as 4. Display "build train station" as the player's move so it's clear.
- If player's numStation = 3, this is the player's first time building a train station, so he/she needs just 1 train card.
  If player's numStation = 2, this is the player's 2nd time building a train station, he/she needs to play and discard 2 cards of the same color.
  If player's numStation = 1, this is the player's 3rd time building a train station, he/she needs to play and discard 3 cards of the same color.
- If the player does not select the right number of cards, display the error message " x cards are needed to build the train station" to guide the player.

- Player plays/discards 1, 2, or 3 cards by clicking on them. Player's playerDeck and discardDeck are updated accordingly.
- Players can select which city to put the station by clicking on gameMap.
- Update gameScreen to display the selected city with the player's color train station so it's visible.
- player.numStations reduced by 1.

3.4 Game End

If any player has 2 or less trains at the end of their turn, each player including this player, gets one final turn. The game then ends. Use method gameOver() to check if any player has <= 2 trains remaining after each play step. gameOver() returns a boolean value TRUE if any player has <=2 trains. This will trigger the final turn for every player.

a. Set gameScreen as non visible and endScreen as visible
b. Call the calculateWinner method which will calculate the final score for each player and display the winning player.

4 Cards, Tickets and Map Creation Algorithm

As described in the last section, at the beginning of the game, the train cards and destination tickets will be initialized as arraylists and values assigned based on the game rules. The cards will be assigned colors. The tickets will be assigned start city, end city, and points.

The game map will also be initialized as HashMap with its cities and routes values assigned the same as those in the actual "Ticket to Ride Europe" board game.

5 Scoring Algorithm

5.1 During the game

Players get points based on how many trains they place along the route. For example, when the player places one of his colored trains on each space, and scores the number of points indicated on the Route Scoring Table for that route's length: placing 1 Train Car gets a player 1 point, placing 2 Train Cars gets a player 2 points, placing 3 Train Cars gets a player 4 points, placing 4 Train Cars gets a player 7 points, placing 6 Train Cars gets a player 15 points, and finally, placing 8 Train Cars gets a player 21 points on a single turn.

Those points are stored at each player's "score" field after each turn.

5.2 At the end of the game

The game ends when any player has less than three train pieces left and every player gets an additional turn including that player. After each player's final turn, the final score is calculated by calling method calcFinalScore().

- First, it is necessary to evaluate the destination tickets: Use the method isTicketComplete(Player, Ticket) to check if the player completes the destination ticket or not. The player's score is adjusted accordingly depending on if the ticket is completed or not. If the ticket is complete, the

player adds the specific number of points for that ticket towards his/her score, however, if the ticket is incomplete, the player loses the specific amount of points on the card and has to subtract it from his/her current score.

- Then the number of train stations that the player has not used is considered. For each train station that a player does not use, he/she will receive 4 points. This one is straightforward, and we can simply use the player's numStations field.
- The last scoring criteria is regarding the European express card. The European express card grants 10 points to the player with the longest continuous route of their specific train color. Use method longestRoute() to check each player's longest route value. The player with the longest continuous route will be given 10 extra points.

5.3 If there is a tie

In the rare circumstance that there is a tie at the end of the game, the person with the most completed destination tickets is crowned as the winner. However, if everyone that is tied has completed the same number of destination tickets, then the winner is determined by looking at which player has the longest continuous path among the players that are tied for the lead.

6 GUI Discussion

Our team has decided to create 3 screens to aid the play of the game. We decided to go with a cartoon theme of Thomas The Train mixed with tan elements to highlight the train concept of the game.

6.1 Start the game

This is our mock-up of our start page (startPanel). If a player needs a refresher on how the game works, they can click the "Game Rules" button, which will display the instructions by calling method drawInstructions(). The "Start Game" button starts the game and transitions it to the game screen, by showing gameScreen Jpanel.



This is our mock-up of the game instructions page which displays the instructions of Ticket to Ride Europe in case the players want to refresh themselves with the game rules.



Beginning of the Game

Before taking their first turn, players must choose which Destination Tickets they will keep from among those they were initially dealt. Each player must keep a minimum of two tickets although they may keep more. Put any Destination Tickets you wish to discard back in the game box without letting any of the other players see them. The Tickets that are put away may either be long routes or regular ones. The tickets you decide to keep are held until the end of the game.

Object of the Game

The object of the game is to score the highest total number of points. Points can be scored by:

- Claiming a Route between two adjacent cities on the map;
- Successfully completing a continuous path of routes between two cities listed on your Destination Ticket(s);
- Completing the Longest Continuous Path of routes to win the European Express Bonus card;
- And for each Train Station kept in reserve at the end of the game.

Points are deducted from the players' total score for each of their Destination Tickets that are not successfully completed by the end of the game.

The Game Turn

The player who has visited the most European countries in his lifetime begins the game, and play then proceeds clockwise. On his turn, a player must perform one (and only one) of the following four actions:

Draw Train Car Cards – The player may draw two Train cards (or just one, if the card he selects is a face-up Locomotive. See **Locomotives** for special rules);

Claim a Route – The player claims a route on the board by playing a set of Train cards from his hand that match the color and quantity of the spaces that make up the route. He places one of his colored trains on each space, and scores the number of points indicated on the Route Scoring Table for that route's length;

Draw Destination Tickets – The player draws three Destination Tickets from the top of the Tickets deck, and must keep at least one of them;

Build a Train Station – The player may build a Station in any city that does not yet have one. To build his first Train Station, the player plays one Train card of any color and places one of his Train Stations on that city. To build his second station, a player must play a set of two cards of the same color, and to build his third station a set of three cards of the same color.

Drawing Train Cards

There are eight types of regular Train cards, in quantities of 12 each, and 14 Locomotive cards. The colors of each type of Train card match the colors of various routes between cities on the board – Purple, Blue, Orange, White, Green, Yellow, Black, and Red.

If a player chooses to draw Train cards, he may draw two cards per turn. Either of these cards can be drawn from the five face-up cards next to the board or from the top of the deck (blind draw). If drawing a face-up card, the player must immediately replace it with a new card taken from the top of the draw pile. If a player selects a face-up Locomotive card, it is the only card he may pick this turn (see Locomotives). If, at any time, three of the five face-up Train cards are Locomotives, all five cards are immediately discarded, and five new cards turned face-up to replace them.

A player may have any number of cards in his hand at any time. When the draw pile is exhausted, the discards are reshuffled into a new draw pile deck. The cards must be shuffled thoroughly, since they will usually have been discarded in sets.

In the unlikely event there are no cards left in the deck, and no discards available to shuffle as a result of players hoarding cards in their hands, a player will not be able to draw Train cards. He must then Claim a Route, Draw Destination Tickets or Build a Station.

Locomotives

Locomotives are multi-colored, and act as wild cards in the game.

Locomotive cards can be played along with any set of cards when claiming a route. They are also vital to claiming Ferry routes (see Ferries).

If a face-up Locomotive card is picked during a card draw, it must be the only card picked that turn. If a Locomotive is turned over as a replacement for a first card drawn during the turn, or if a Locomotive is available face-up but not picked up as the first (and only) card, it cannot be selected as a second card.

However, if a player is lucky enough to get a Locomotive from the top of the deck in a blind draw, it still counts as a single card and he may still draw a total of two cards that turn.

Claiming Routes

A route is a set of continuous colored spaces (in some instances, gray spaces) between two adjacent cities on the map. To claim a route, a player must play a set of Train cards whose color and quantity match the color and number of spaces of the chosen route.

Most routes require a specific set of colored cards to claim them. Locomotives can always act as a stand-in for any given color (See example 1).

Routes that are Gray can be claimed using a set of cards of any one color (See example 2).

When a route is claimed, the player places one of his plastic trains on each of the spaces of the route. All the cards from the set used to claim the route are then discarded. The player then immediately records his score by moving his Scoring Marker the appropriate number of spaces along the Scoring Track, per the Route Scoring Table on page 7.

A player may claim any open route on the board. He is never required to connect to any of his previously played routes.

A route must be claimed in its entirety during a single turn. For example, you cannot lay two trains on a three-space route and wait for the next turn to lay the third train.

No more than one route may be claimed in a given player's turn.

Example 1: *To claim a Blue route that is three spaces long, a player may play any of the following card combinations: three Blue cards; two Blue cards and a Locomotive; one Blue card and two Locomotives; or three Locomotives.*

Example 2: *A Gray route that is two spaces long can be claimed by playing two Red cards; one Yellow and a Locomotive; or two Locomotives.*

Double-Routes

Some cities are connected by Double-Routes. These are routes whose spaces are parallel and equal in number from one city to the other. One player can never claim both routes between the same cities during the course of the game.

Be aware of routes that are partially parallel to each other but are linked to different cities. These are not double-routes.

Important Note: In two or three player games, only one of the Double-Routes can be used. A player can claim either of the two routes between cities, but the other route is then closed to the other players for the remainder of the game.

Ferries

Ferries are special Gray routes linking two adjacent cities across a body of water. They are easily identified by the Locomotive icon(s) featured on at least one of the spaces making the route.

To claim a Ferry Route, a player must play a Locomotive card for each Locomotive symbol on the route, and the usual set of cards of the proper color for the remaining spaces of that Ferry Route.

Claiming the Ferry Route from Algeciras to Palermo requires four Train cards of any one color and two Locomotives.

Tunnels

Tunnels are special routes that are easily identified by the special tunnel marks and outlines surrounding each of their spaces.

What makes a tunnel special is that a player is never quite certain just how long the route he is trying to claim will be!

When attempting to claim a Tunnel route, a player first lays down the number of cards required by the length of the route. Then the three top cards from the Train draw pile are turned face-up. For each card revealed whose color matches the color of the cards played to claim the Tunnel, an additional card of the same color (or a Locomotive) must now be played from his hand. Only then can the player successfully claim the Tunnel route.

Two parallel tunnels – one Black and one White – link Madrid to Pamplona.

If the player does not have enough additional Train cards of the matching color (or does not wish to play them), he may take all his cards back in his hand, and his turn ends.

At the end of the turn, the three Train cards revealed for the Tunnel are discarded.

Remember that Locomotives are multi-colored wild cards. As such, any Locomotive card drawn from the top of the Train draw pile during an attempt to go through a tunnel will automatically match the color of the train cards played on the route, and force the player to play an additional card.

If a player attempts to go through a Tunnel using Locomotive cards exclusively, he will only have to play additional cards (which must be additional Locomotives in this case) if Locomotives show up among the three cards drawn for the Tunnel.

In the rare event there is not enough cards available in the draw and discard piles to reveal 3 cards and determine the effect of the tunnel on a player, then only those cards that are available are revealed. If, as a result of players hoarding cards, there are none to be revealed, a tunnel can be claimed without risking additional cards.

Example 1: Playing 2 red cards, 1 red card revealed : 1 more red needed

Example 2: Playing 2 green cards played, 1 loco revealed : 1 more green needed

Example 3: Playing 2 loco played, 1 loco revealed : 1 more loco needed

Drawing Destination Tickets

A player can use his turn to draw additional Destination Ticket cards. To do so, he draws three new cards from the top of the Destination Ticket Deck. If there are less than three Destination Tickets left in the deck, the player only draws the cards that are available.

A player drawing Tickets must keep at least one of them, but may choose to keep two, or all three. Any drawn Ticket not kept in hand is placed at the bottom of the Destination Ticket Deck. Tickets drawn and not immediately discarded must be kept until the end of the game. They cannot be discarded during a later Ticket draw.

The cities listed on a Destination Ticket represent travel goals for the player; they can result in a bonus or a penalty. If, by the end of the game, a player has created a continuous path of his color plastic trains between the two cities named on a Destination Ticket he holds, he scores the additional points indicated by the Point Value on the Ticket. If he has failed to complete a continuous path between those cities, he deducts the Point Value on the Ticket from his total score.

Destination Tickets are kept secret from other players until the game's final scoring. A player may have any number of Destination Tickets during the game.

Building a Train Station

A Train Station allows its owner to use one, and only one, of the routes belonging to another player, into (or out of) that city to help him connect the cities on his Destination Tickets.

Stations may be built on any unoccupied city, even if it currently has no claimed routes into it. Two players may never build a Station in the same city.

Each player may build a maximum of one Station per turn, and three Stations throughout the course of the game.

To build his first Station, a player plays and discards one Train card from his hand, and places one of his colored Train Stations on the chosen city. To build a second Station, the player must play and discard a set of two cards of any one color; and to build his third, a set of three Train cards of any one color. As usual, you can replace any number of cards by Locomotives.

If a player uses the same Station to help connect cities on several different Tickets, he must use the same route into the city with the Station for all of those Tickets. The Train Station owner does not need to decide which route he will use until the end of the game.

A player is never required to build any Stations. For each Station a player has not used, four points are added to his score at the end of the game.

Game End

When any one player's stock of colored plastic trains gets down to two trains or less at the end of his turn, each player, including that player, gets one final turn. The game then ends and players calculate their final scores.

For less plastic trains as a player's stock signal the final round of game of the game.

Calculating Scores

Players should have already accounted for the points they earned as they completed various routes. To make sure no mistakes were made, you may want to recount the points for each player's routes.

Route Length	Points Scored
1	1
2	2
3	4
4	7
5	10
6	15
8	21

When a player claims a route, he immediately scores the number of points indicated on the Route Scoring Table for the length of the route he just claimed.

Players must then reveal all of their Destination Tickets. The value of successfully completed tickets is added to their total score. The value for any incomplete Tickets is deducted from their total score.

Remember that each Station played allows its owner to use one (and only one) route belonging to another player into that City for the purpose of completing a Destination Ticket. If a player uses the same Station to help connect cities on the paths of several different Destination Tickets, he must use the same route into or out of the city with the Station for all Tickets.

Add to the score of each player four points for each unplayed Station they still have in reserve.

Finally, give the 10 point bonus for the European Express to the player(s) who have the Longest Continuous Path on the board. When evaluating and comparing path lengths, only take into account continuous lines of plastic trains of the same color. A continuous path may include loops, and pass through the same city several times, but a given plastic train may never be used twice in the same continuous path. Stations, and the opponents' routes they may provide access to, do not count for the purpose of computing paths and claiming the longest one. If several players are tied for the longest path, they each receive the 10 points bonus from the European Express card.

The player with the most points wins the game. If two or more players are tied with the most points, the player who has completed the most Destination Tickets is the winner. If still tied, the player who used the least number of Stations is declared the winner. In the unlikely event players are still tied, the player with the European Express bonus card wins.

Calculating Final Score

Components

- 1 map of Europe
- 240 Colored Train Cars (45 each in Blue, Red, Green, Yellow and Black, plus some extra replacement cars in each color)
- 15 Colored Train Stations (three each, matching the trains' colors)
- 158 Illustrated cards including:

110 Train Car cards : 12 of each, plus 14 Locomotives

40 Destination Ticket cards

8 long routes with a blue background

60 regular routes

1 Summary card

1 European Express Bonus for the Longest Continuous Path

- 1 Wooden Scoring Markers (1 each of the 5 players' colors)
- 1 Rules booklet
- 1 Days of Wonder Online access number (located on back of Rules)

Important Note: Players familiar with the first game in this series, Ticket to Ride™, should focus their attention on the following new features: Ferries, Tunnels and Train Stations.

Setting Up the Game

Place the board in the center of the table. Each player receives a set of 45 Colored Train Cars, a matching set of three Colored Train Stations and the corresponding Scoring Marker. Each player places his Scoring marker on Start ❶ on the Scoring Track running along the map's border. Throughout the game, each time a player scores points, he will advance his marker accordingly.

Shuffle the Train cards and deal a starting hand of four cards to each player ❷. Place the remaining deck of Train cards near the board, then turn the top five cards from the deck face up, and lay them one next to the other ❸.

Place the European Express Bonus and the Summary card, face up, next to the board, as a reminder to the players ❹.

Take the deck of Destination Tickets and separate the long routes (the six Destination Tickets with a blue background) from the regular routes; Shuffle the long routes, and randomly deal one to each player ❺. Put any remaining long routes back in the game box without letting anyone see them.

Now shuffle the regular Destination Tickets – all short routes with a plain background –, deal three to each player ❻, and place the remainder in a draw pile face down, next to the board.

You are now ready to begin.

6.2 Play the game

Below is our main game screen mockup - the board that will be active for the majority of the game:

- The middle of the screen is the current game map which displays all of the routes available, the purchased routes, and also the train stations that have been built
- The top left corner of the screen is the player status board which lists each player's color, current score and longest longest continuous train car route.
- On the right of the player status board is the destination ticket draw pile from which the player can draw new destination tickets
- On the right of the destination ticket cardback are the five face up cards which the player is eligible to draw from.
- On the very top right corner is the cardback for the facedown draw pile of train cars which the player can also draw from
- On the left side, below the player status board is the prompt message box which lists the current player, type of player's move, and notifies a player with a message whenever necessary.
- On the bottom left corner of the screen is the player move box. It has 4 buttons from which the player can choose what move he/she makes for the turn.
- On the bottom, next to the player move box is the current player's inventory of trains, depicted by the train symbol with a counter; stations, depicted by the station outline and counter; tickets; and the current card inventory of the player, displayed with cards and however many the player holds for each color. Once a player starts his/her turn, the bottom of the screen automatically displays his/her current possessions. This helps the

player to visualize his/her cards and trains inventory, and to plan his/her move to complete the route on his/her tickets.



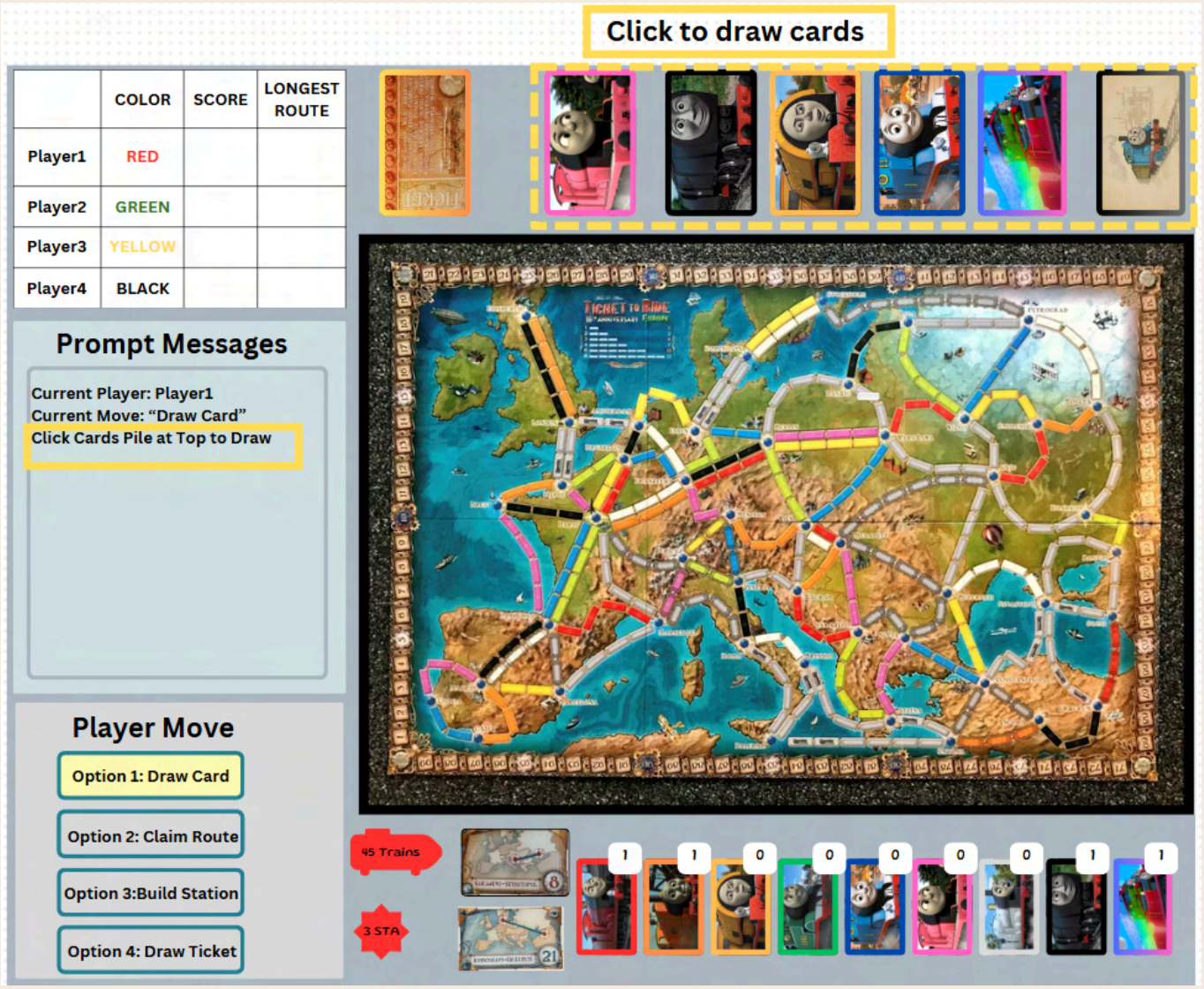
During the game, we use mouseListener to monitor the player's mouse clicks and take that as input to the game logic.

- The player first selects which of the 4 options he/she wants to take, by clicking on 1 of the 4 different player move options. The selected option will be highlighted on the screen and the prompt message window will display the selected option to echo the

player's move. See image below with the changes in the yellow highlighted box.



- If the player selects "draw cards" or "draw tickets", the program will listen to mouse click of the player when he/she clicks on the house train cards (hidden or face up ones), or house tickets. The selected house train cards or house ticket cards will be added to the bottom of the screen where the player's inventory cards are.



Assume the player draws 2 cards from the hidden pile and the 2 cards are both red and added to the player's cards inventory at the bottom of the game screen. The number of red cards is updated from the previous 1 to 3 now, as shown in the mock up screen below, with the card number "3" yellow highlighted.

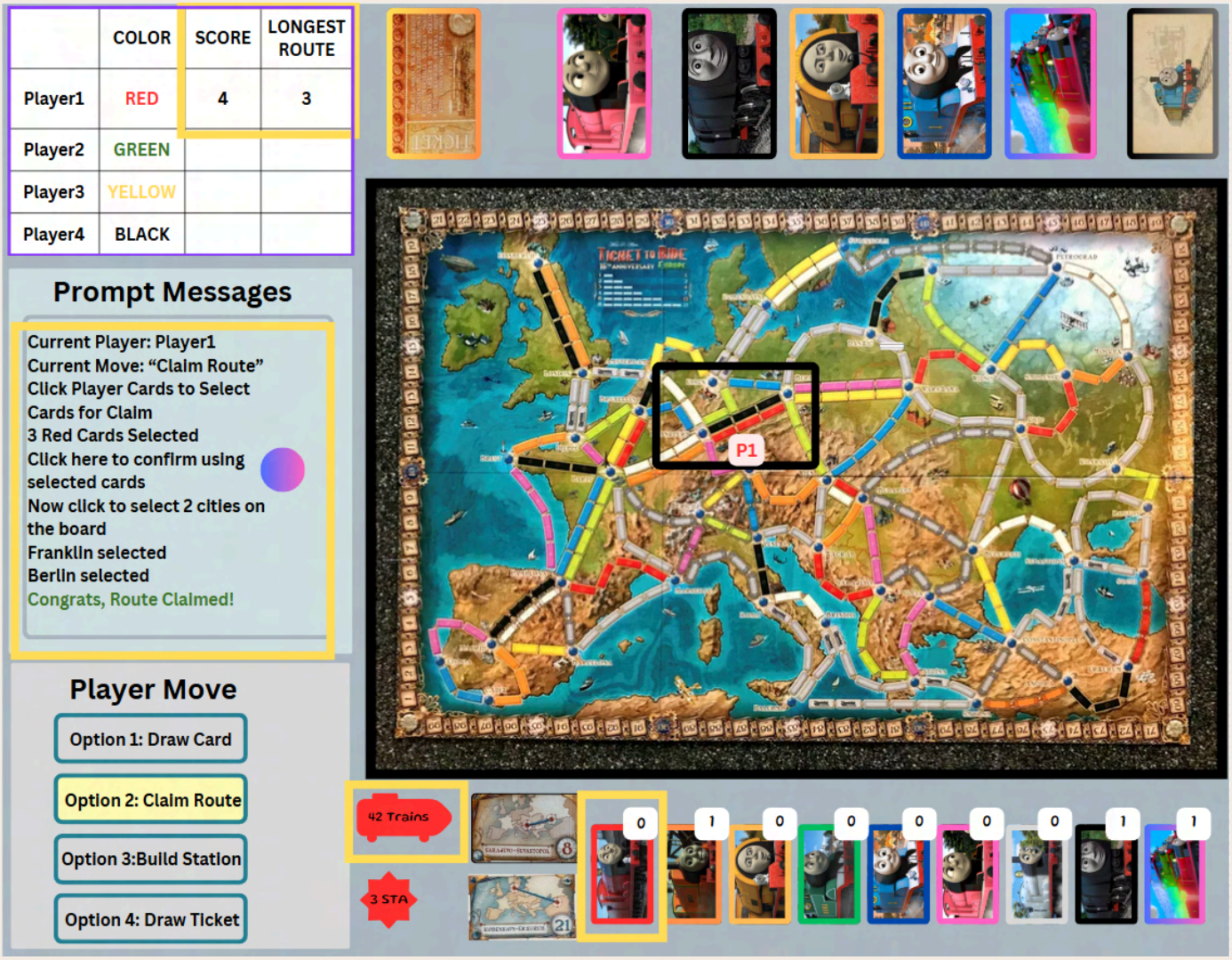


- If the player selects "claim route", the prompt message window will display "select the train cards to claim". The program will use mouse click to monitor the player's click on his/her own inventory train cards. The cards selected by the player will be moved slightly upwards to highlight how many and what color cards are selected by the player.
  Assume player1, at his/her next turn, wants to use the 3 red cards he/she has to claim a route. He/she clicks on the 3 red

cards and the mock up screen is as below. The 3 selected cards are shown as "popped up" from the rest of his/her inventory cards, to highlight those are the selected cards. In the Prompt Messages window, the screen shows a "Confirm using selected cards" button that the player clicks to confirm that he/she is done with card selection and ready to move on to select the cities/route.



- Once the player clicks "Confirm using selected cards", the program monitors the player's mouse click on the game board. The Prompt Messages shows "Now click to select 2 cities on the board" to guide the player his/her next action. When the player clicks on a city on the gameScreen, the program will register that as a city that the player wants to claim a route on. The program also displays a prompt message "citynamexxx is selected". The player needs to click and select the destination city in the same way. If the player has the right number of cards with the right color, the prompt message window will display "Congrats! Route claimed!" The program will also add the player's name along the claimed route so it's visible to all players. The route's color to be updated to the player's color - representing that the player's train cars are placed on the route. The player status board at top left corner is updated accordingly.



- Once the player clicks "build station", the program monitors the player's mouse click on the game board. The program displays a prompt message "city to be selected". When the player clicks on a city on the gameScreen, the program will register that as the city that the player wants to build a station on. If the player is successful in building the station, the player's name will be displayed next to the city, so it's visible to all players.

	COLOR	SCORE	LONGEST ROUTE
Player1	RED	4	3
Player2	GREEN		
Player3	YELLOW		
Player4	BLACK		

Prompt Messages

Current Player: Player1
Current Move: “Build Station”
Click to select city on the board

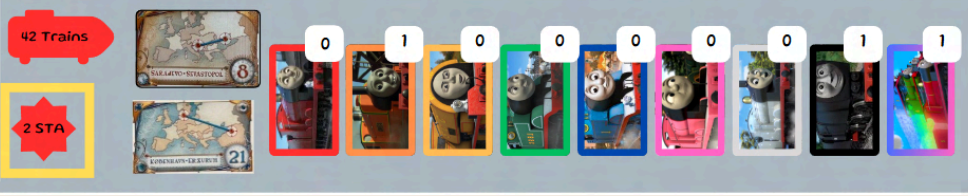
Paris selected

Congrats, Station Built at Paris!

Player Move

- Option 1: Draw Card
- Option 2: Claim Route
- Option 3:Build Station
- Option 4: Draw Ticket





6.3 End of the game

When the game ends, the image depicted below will be displayed. The winner will be displayed. Furthermore, the players and their respective scores will be presented next to the players. The Prompt Messages window displays the winner information and also displays "Click here to play the game again" with a click button next to it. The player can click that to play the game again.



7 Program Testing Methodology and Test Data

We developed a few tests that need to be completed in order to confirm that the game works as expected and we will run these tests before the code gets pushed into the main branch.



7.1 Early stages/Game setup:

- ☐ Make sure that each train card is created correctly when starting a game
- ☐ Make sure that each destination ticket card is created correctly when starting a game
- ☐ Ensure that every player is created and has correct attributes
- ☐ Ensure that the game map is created correctly with each city and routes. Ensure the game map can be plotted on the game screen appropriately. We will start with a smaller map with less cities and routes for testing purposes, before we build up to the full game map.

7.2 Start of Gameplay Setup

- ☐ Check that each method works as expected, start from the simple ones like drawing from house cards, before getting to the more complicated ones.
- ☐ Ensure that all objects or variables are used correctly
- ☐ Make sure that everything is error free and no carefree mistakes exist
- ☐ Ensure that wildcard (locomotive) functions correctly
- ☐ Check that the cards and tickets, their colors, images, points, starts and destinations are stored correctly
- ☐ Ensure that mouse listener works correctly and as intended in the game logic

7.3 Gameplay Testing

- ☐ Create and test the game logic first without the graphics. We will use simple text input and output at this stage, just to make sure the underlying game logic works correctly, before we add in the graphics part.
- ☐ Make sure our methods and functions work correctly for gameplay
- ☐ Test that each player can utilize all the functions and moves during gameplay
- ☐ Debug any problems that occur when making our panel
- ☐ Check the visuals of our game
- ☐ Ensure that everything is in the correct position on the game board
- ☐ Refresh game board after each player's turn. Make sure that the player's cards and station number stay updated.

7.4 End of Game

- ☐ Test to make sure that end condition is correct
- ☐ Ensure that the game ends when it is meant to end
- ☐ make sure that each player still gets the final turn
- ☐ Ensure that points are calculated correctly for each player and the correct winner is displayed

7.5 Test Data

The following test data are planned to verify that the program will work following the game logic. The test data are the special cases that could potentially be handled incorrectly. Those cases will be used as part of the debugging.

1) Game set up:
    a) Test that all cards, tickets, stations, players, cities, routes are assigned correctly by using Println() to check values stored in the corresponding arraylist.
2) Draw train car cards:
    a) if a player selects a face up locomotive card, he/she is only allowed to draw 1 card instead of 2.
    b) If a locomotive is available face up but not picked by the player, the player cannot pick that face up locomotive card as his/her second card.
    c) If 3 of the 5 face up cards are locomotives, all the 5 need to be discarded (put to discardDeck arraylist) and 5 new cards taken from the hidden deck. This will be handled through the method checkFaceUpfor3LOC(). We can do a test run with more locomotive cards (than the actual game) to test the game logic.
3) Claim route:

a) A player wants to claim a route while he/she doesn't have the right color or right number of train cars for the route. In this case, the Prompt Message window should display "cards are not correct to claim" and allow the player to select the cards again.
b) Test that the locomotive can work as a stand-in for any other color.
c) Test that a gray route can be claimed by a set of cards with any one color.
d) Test that if a route is already claimed by one player, the same route cannot be claimed by a different player. This is implemented by checking the OwnedBy field if it's empty or not.
e) A player is not allowed to claim double routes between 2 cities. Double routes are same distance routes between 2 cities. In the program, this is identified through hasDoubleRoute of the EdgeRoute class. Test a scenario that if a player already claimed one of the double routes, he/she will not be allowed to claim the other one of the double routes.
f) Claim a tunnel route: test a scenario when a player wants to claim a tunnel route and make sure the game logic is followed. The player needs to draw 3 train cards from the deck hidden pile and if any drawn card matches the color of the tunnel route, the player needs to use additional card(s) to claim the tunnel route. If the player doesn't have enough cards, he/she cannot claim the tunnel route and his/her turn is over.

4) Draw tickets:
    a) Test that a player, after drawing 3 destination tickets, can choose to keep at least 1, or 2, or all 3 of the destination tickets. For the destination tickets that he/she doesn't want, those would be added to the bottom of the ticketDeck arraylist.
5) Build train station:
    a) Test when a player wants to build the 1st, 2nd and 3rd train station. Use the numStation field of the player class to distinguish if this is the player's 1st, 2nd or 3rd station. Make sure that the program also deduct 1, 2, or 3 train cards from the player's cards inventory.
    b) Test that when a city already has a station built by one player, the program will not allow one more station to be built. The city class has a field "hasStationOwner". Once a city's "hasStationOwner" is true, that city shall not allow more stations to be built.
6) Tracking player's turn
    a) A player should be only given one turn. If a player claims a route, his/her turn is over. This is tracked by the player's currentMove field. For the active player, his/her currentMove is set to 1/2/3/4 depending on what option he picks. Once he/she is done with the current play, the player's currentMove will be set to 0, which means that any screen click of the player should be ignored as it is not his/her turn. Test if a player continues to play, his/her

screen click is disabled and the program should move onto the next player.

7) Game End:
    a) Test that the program can correctly identify when the game is over. Test a game play to see if the program can capture the game over status correctly.
    b) Test that the IsTicketComplete() method runs correctly. Start with a short ticket that is relatively easier to check completion or not. Test the case when the player has built a station in a city along the ticket.
    c) Test that the longestRoute() method runs correctly and the program returns the correct maximum length for each player.
    d) Test that the winner is selected correctly, when there are ties.

8 Foreseeable Problems and Issues



8.1 Game Map Graphics

To allow graphic user interaction during the game play, the gamp map needs to include all components drawn based on their coordinates: cities, routes and train stations. We cannot use a static image for the game map, as the game map needs to be constantly updated depending on the player's move. For example, if a player claims a route, the route color needs to be updated to reflect its owner. The cities need to be clickable so the player can click on the game map to select the city for station building or for route claiming. The components of the game map, cities/routes/stations, will be updated during game play. We will start from a relatively small game map with fewer cities to test the game logic, before we move to the full Ticket to Ride Europe game map.

8.2 Monitor Player's Mouse Click

To allow graphic user interaction during the game play, the player's mouse click needs to be monitored to track what he/she wants to do next. The player may click on his cards, house cards, house tickets, cities. The program needs to track all the various mouse clicks, register that and apply corresponding game logic. The graphic user interface could be a challenge for this project.

8.3 Locomotive Incorporation

The incorporation of the locomotive train cards adds another layer of complexity to the coding process. The locomotive train card can represent any color, allowing for greater flexibility in purchasing railroads. However, this independence makes it challenging to ensure that all possible moves or options are considered within the code. The logic must account for various scenarios where players choose to use the locomotive effectively. Mismanagement in this area can lead to missed opportunities or confusion over the rules, ultimately impacting gameplay.

8.4 Check Ticket is Completed or Not

To check if a player has completed his/her ticket, we need to determine if a continuous path exists between the two cities on the ticket using the player's claimed routes. In addition, we need to account for if any station was built by the player, which would allow him/her to use the route.

We also need to think about the algorithm to get the longest route for each player.

8.4 Handling Multiple Players

Handling multiple players in *Ticket to Ride Europe* presents several challenges that require careful consideration. Each player has their own set of cards, trains, train stations, and destination tickets which means the code must effectively manage and track these individual inventories. Additionally, interactions between players, such as competition for routes, can complicate gameplay. This complexity necessitates a robust system to ensure that player actions do not interfere with one another, which could lead to confusion or errors during play.

8.5 Game End Conditions

Determining when the game is over is another small yet critical problem. *Ticket to Ride Europe* has specific conditions that trigger the end of the game, primarily when a player reaches a certain number of trains left. The code must accurately monitor player train inventories and recognize these conditions without delay. Failure to recognize the end game condition could result in an unsatisfactory gaming experience, as players might be uncertain about the game's status and everything may freeze.

8.6 Debugging Challenges

Debugging small issues in our code will likely be our greatest challenge throughout the development process. Given the multifaceted nature of the game, it may require considerable time and effort to identify and resolve minor problems. Oversight in one area can lead to cascading errors in others, especially in a game with many interdependent components like *Ticket to Ride Europe*. As the codebase grows, the potential for errors increases, making it essential to implement thorough testing strategies early in the development phase to catch these bugs before they compound.